Continental
The Future in Motion

Security in Applications/Software Components

Florin Anton

www.continental.com/ro-ro

Agenda

1. What is secure coding?
2. Why is secure coding important?
3. Best practices for secure coding
4. Secure coding Guidelines
   I. CWE, CERT C, MISRA – Classic Autosar
   II. AUTOSAR C++14 Coding Guidelines – Adaptive Autosar

What is secure coding?

Secure coding, also referred to as secure programming, involves writing code in a high-level language that follows strict principles, with the goal of preventing potential vulnerabilities (which could expose data or cause harm within a targeted system).

Secure coding is more than just writing, compiling, and releasing code into applications. To fully embrace secure programming, you also need to create a secure development environment built on a reliable and secure IT infrastructure using secure hardware, software, and services and providers.

Why is secure coding important?

- **Access control**, which includes authentication and authorization, is one of the basic building blocks of protecting your system.
- **Enforcing strong encryption** is another important aspect of keeping your system secure. There are many readily available libraries to help you implement encryption, thus requiring minimal custom code be written. It's, however, important to only use standard algorithms and libraries. You should also ensure that whenever FIPS compliance is required only validated libraries are used.
- **Secrets management** is another important security measure. Whether or not you choose to use one of the many available tools to help you manage secrets, you should never hardcode or upload secrets such as passwords or access keys to code repositories.

The above measures will protect your system and are thus the first line of defense, but it's also critical to make your code itself more secure.

Best practices for secure coding

Code minification and obfuscation: Making your code harder to access, and by extension harder to read, can deter potential attackers. In the JavaScript world, a common practice is to minify code. Minification removes white space and line breaks from your code. And while it is and is really intended to enhance performance by reducing the footprint of code files, it has the added benefit of making exposed code much harder to read. Another similar, more effective technique is code **obfuscation**, which turns human-readable code into text that is difficult to understand.

Avoiding shortcuts: It can be tempting for developers to want to take shortcuts to release code into production faster, but this could have serious security implications. For example, attacks often occur when hardcoded credentials and security tokens are left as comments. This information should be cleaned up long before your apps are released. But as your code base gets bigger and there is mounting pressure to deliver working code on increasingly tight release schedules, the likelihood of security gaps goes up.

Automated scanning & code reviews

Avoiding components with known vulnerabilities: While open-source components and libraries, often consumed as packages, can save developers time and energy, they are also a common entry point for malicious actors and a great source of vulnerabilities and potential exploits. Refraining from using those components with known vulnerabilities and constantly monitoring for new vulnerabilities throughout the development process in the components you use will help you maintain the integrity of your code.

Auditing & logging: Software with sufficient logging and monitoring will allow you to detect potential incidents when your code is deployed in a production environment.

Secure coding Guidelines
CWE, CERT C, MISRA – Classic Autosar

Software security is a top concern today. You can't risk any security vulnerabilities — particularly if you're developing software for embedded systems. And that means your code needs to be secure and free of coding errors.

Ways to Ensure Secure Code in C:

- CWE You can identify security weaknesses from the Common Weakness Enumeration (CWE).
- CERT C
- ISO/IEC TS 17961:2013 "C SECURE"
- MISRA C

Secure coding Guidelines
AUTOSAR C++14 Coding Guidelines – Adaptive Autosar

The AUTOSAR C++ 14 guidelines are an update for MISRA C++ 2008, providing coding guidelines for modern C++ as defined by ISO/IEC 14882:2014.

The automotive industry has undergone radical changes in the last decade. New functionalities introduced to modern cars have fundamentally changed them into computing centers. This is reflected in the development of the platforms that are used for building automotive software, specifically AUTOSAR.

The classic approach, which assumed static configuration at compilation time, no dynamic memory allocation, and predominance of the C language, is no longer sufficient. New features like these require a lot of advanced code:

- Lane-keeping assist
- Automatic emergency braking
- Blind-spot monitoring
- V2X communication (vehicle-to-vehicle and vehicle-to-infrastructure communication)
- Continual over-the-air updates
- High definition multimedia
- AI computing, image recognition

Thank you!